

ONLINE RECIPE SHARING PLATFORM

Java Web Project • Review 1

A role-based community for publishing, discovering, rating and saving recipes.

WEB-BASED

JSP + SERVLETS

JDBC + MYSQL



Prepared for Galgotias University • 2026

Why the web-based option is the lower-risk choice

Decision based on the project idea, the two-day build window, and the supplied Web-based marking rubric.

Decision

Build it as a Java Web application.

- Fits the Instagram / Reddit-style community UI naturally.
- JSP + Servlets make request/response and sessions easy to demonstrate.
- JDBC maps cleanly to the required database integration.
- Responsive HTML/CSS can be shown directly in browser screenshots.

LOWER BUILD RISK

Review 1 rubric alignment

Problem Understanding & Solution Design **8** marks

Core Java Concepts **10** marks

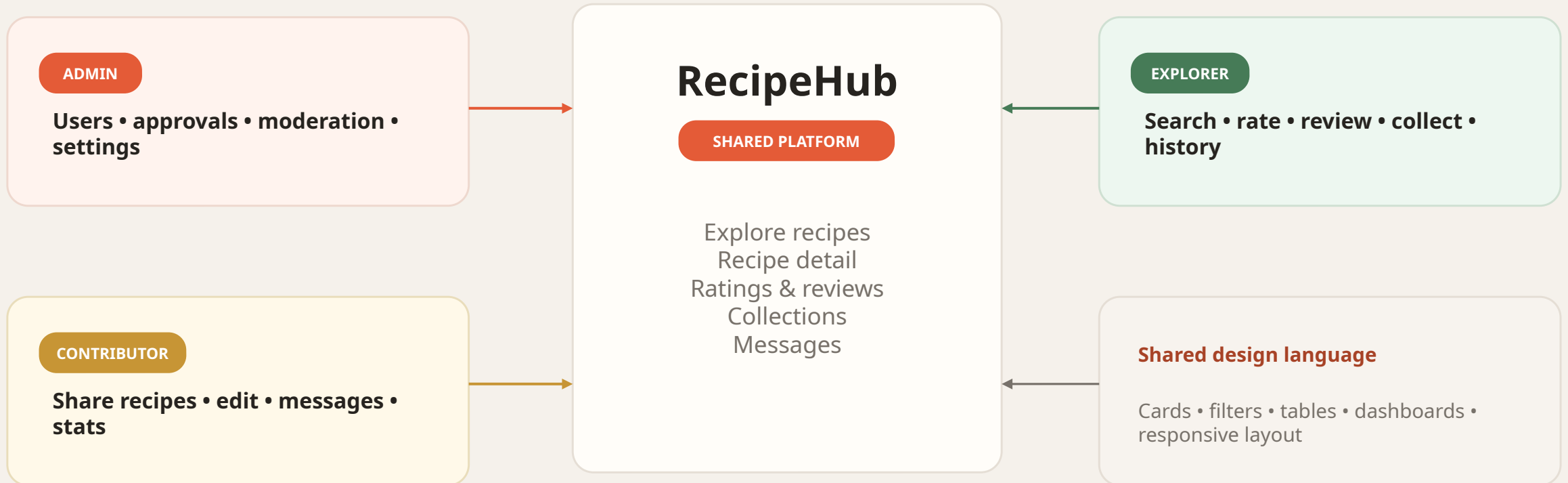
Database Integration (JDBC) **8** marks

Servlets & Web Integration **7** marks

Review 1 total **33**

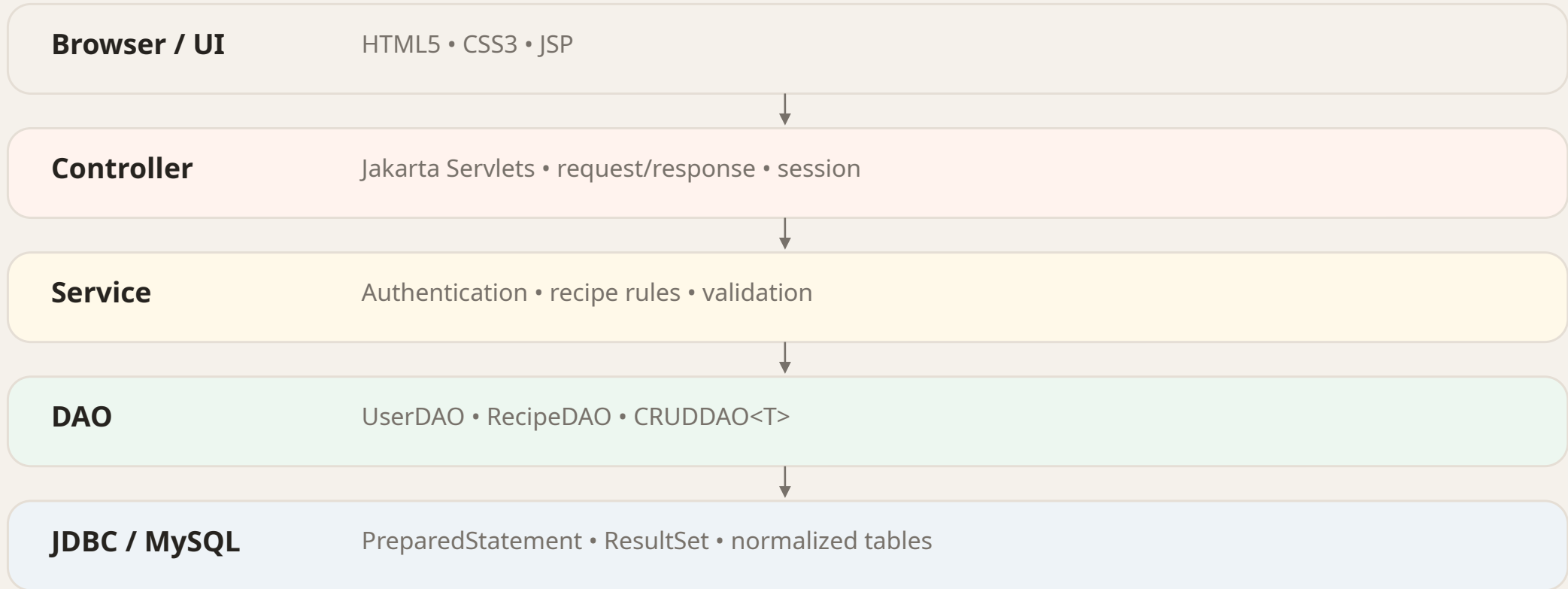
One platform, three role experiences

Shared feed and recipe pages, with role-specific dashboards and permissions.



Simple MVC-style Java web architecture

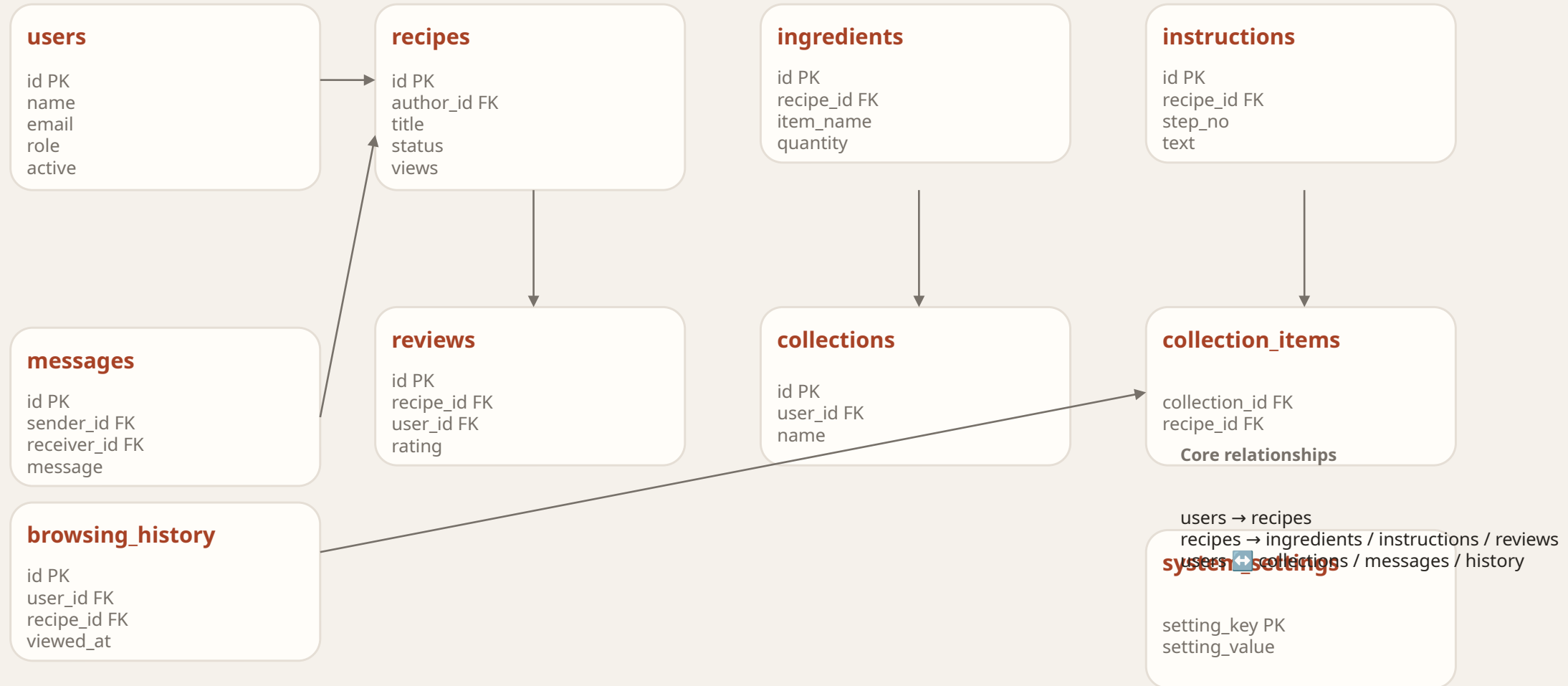
Keep the layers obvious so the reviewer can trace one request from browser to database.



Cross-cutting: AuthFilter • application exceptions • session user • role checks

Normalized schema for the platform

The design separates users, recipes, recipe parts, reviews, collections, messages, history and system settings.



A social-feed style interface without copying a specific platform

Warm editorial cards, simple navigation, clear hierarchy and mobile-safe layouts.

SIGN IN

Good recipes deserve good company.

Email

Password

Sign in

EXPLORE RECIPES

What's cooking today?

Search recipes, cuisines, ingredients...

Creamy Garlic Pasta

★ 4.8 • 128 views

Masala Vegetable Toast

★ 4.5 • 94 views

Design principles: consistent cards • strong whitespace • responsive grid • visible role context

The requirements map directly into four dashboard areas

Each dashboard emphasizes the actions that belong to that role instead of putting every feature into one screen.

ADMIN

Control center

- User management table
- Recipe approval queue
- Content moderation
- System settings

CONTRIBUTOR

My kitchen

- Personal recipe management
- Interaction history
- Views / ratings / feedback stats
- Profile management

EXPLORER

My collection

- Browsing history
- Ratings and reviews
- Recipe collections
- Profile & preferences

SHARED

Recipe experience

- Search + filters
- Recipe detail
- Save / collection action
- Community feedback

What is already structured in the repository

The goal is to submit a coherent architecture, schema, connectivity layer and UI plan now, then expand the same structure for Review 2.

Problem / solution design

Role map + feature map + navigation plan

READY**Core Java**

Models, enums, generic DAO contract, custom exceptions, service layer, async StatsService

READY**Database / JDBC**

MySQL schema, foreign keys, indexes, PreparedStatement DAOs, connection utility

READY**Servlets / Web**

Login, logout, feed, recipe detail, dashboards, AuthFilter, JSP views

READY

Review 1 deliverables: presentation + GitHub repository + README + database files + project structure

Two focused build days, then incremental completion

Do not redesign the project after Review 1. Keep this structure and add the remaining CRUD operations and final business features.

DAY 1

Build foundation

- Create Maven project
- Run schema + seed SQL
- Verify JDBC connection
- Implement login / sessions
- Wire feed + recipe detail
- Commit to GitHub

DAY 2

Polish + submit

- Finish dashboard screens
- Add screenshots / diagrams
- Clean README + .gitignore
- Run a smoke test
- Prepare presentation PDF/PPT
- Submit before deadline

REVIEW 2

Finish functionality

- Recipe CRUD
- Approve / reject
- Ratings + reviews
- Collections + history
- Messaging + stats
- Testing + code cleanup

Deadline to protect: **Saturday, 10 October 2026**

Review 2 deadline: **Sunday, 15 November 2026**